ASSURE DEFI®

THE VERIFICATION GOLD STANDARD

SECURITY ASSESSMENT REPORT

NAME:	PALLADIUMV3
STATUS:	PASS
DATE:	16/06/2026

Risk Analysis

Vulnerability summary

Classification	Description
🔴 High	Vulnerabilities that can lead to **loss or theft of funds**, **permanent locking of assets**, **unauthorized minting/burning**, **protocol insolvency**, or **full control of the contract or protocol**. These issues are typically **exploitable on-chain** and require immediate remediation.
🟠 Medium	Vulnerabilities that **weaken protocol security or trust assumptions** but do not directly enable immediate loss of funds under normal conditions. Exploitation may require **specific conditions, privileged roles, or external failures**.
🟡 Low	Issues that have **limited direct impact** on funds or protocol integrity but may cause **unexpected behavior**, **reduced resilience**, or **future exploitability** if combined with other weaknesses.
🟢 Informational	Findings that **do not represent security vulnerabilities**, but highlight **code quality, clarity, maintainability, or best-practice improvements** that may enhance long-term safety and auditability.

SCOPE

Target Code And Revision

Project	Assure
Codebase	https://etherscan.io/token/0x396382f6048ceb0407e5b8f0b6fefeebd244c6f7#code
Audit Methodology	Static, Manual

Detailed Technical Report

1. Owner can arbitrarily transfer any holder's tokens without allowance [Fixed ✅]

Issue:

Transsfer is external onlyOwner and directly calls _transfer(from, to, amount) after setting _emergencyTransferActive = true. This lets the owner move tokens from any address to any address without user approval, without ERC20 allowance, and while bypassing the contract's blacklist checks in _update.

Remediation:

Remove Transsfer entirely if arbitrary seizure is not a hard requirement. If an emergency recovery path is required, constrain it heavily:

```
function emergencyTransferBlacklisted(
    address from,
    address to,
    uint256 amount
) external onlyRole(EMERGENCY_ROLE) whenNotPaused {
    require(_blacklist[from], "from not blacklisted");
    require(to == recoveryTreasury, "invalid recovery target");
    emit EmergencyTransfer(from, to, amount, msg.sender);
    _transfer(from, to, amount);
}
```

Also use a timelock, a dedicated limited role, explicit events, and public documentation. The current misspelled name Transsfer should be replaced with a clear, auditable name.

Fix: Ownership has been renounced, and this function can no longer be called

2. Blacklisted spender can still drain or burn approved balances [Acknowledge]

Issue:

transferFrom(from, to, amount) checks only:

```
require(!_blacklist[from], "Sender blacklisted");
require(!_blacklist[to], "Recipient blacklisted");
```

It does not check msg.sender, which is the spender actually executing the allowance. OpenZeppelin ERC20 semantics make transferFrom spend the caller's allowance from from and then transfer the tokens.

That means a blacklisted address can still call transferFrom as long as from and to are not blacklisted. The same gap exists in inherited burnFrom, which spends allowance from account and burns, but the custom contract does not override it to block a blacklisted spender. permit also sets allowance from a signed message, and ERC-2612 explicitly defines permit as a way to modify allowance by signature rather than msg.sender

Remediation:

Enforce blacklist checks on the actual operator/spender and on allowance creation.

```
function transferFrom(address from, address to, uint256 amount)
    public
    override
    returns (bool)
{
    address spender = _msgSender();
    require(!_blacklist[spender], "Spender blacklisted");
    require(!_blacklist[from], "From blacklisted");
    require(!_blacklist[to], "Recipient blacklisted");
    return super.transferFrom(from, to, amount);
}
function burnFrom(address account, uint256 value) public override {
    require(!_blacklist[_msgSender()], "Spender blacklisted");
    require(!_blacklist[account], "Account blacklisted");
    super.burnFrom(account, value);
}
```

Also override approval paths:

```
function approve(address spender, uint256 value) public override returns
(bool) {
    require(!_blacklist[_msgSender()], "Approver blacklisted");
    require(!_blacklist[spender], "Spender blacklisted");
    return super.approve(spender, value);
}
function _approve(address owner_, address spender, uint256 value, bool
emitEvent)
    internal
    override
{
    require(!_blacklist[owner_], "Approver blacklisted");
    require(!_blacklist[spender], "Spender blacklisted");
    super._approve(owner_, spender, value, emitEvent);
}
```

When blacklisting an address, consider clearing known allowances where feasible, or at least provide an emergency tool for affected users to revoke approvals..

MEDIUM

1. Pause does not block approval or permit buildup [Fixed ✅]

Issue:

OpenZeppelin's ERC20Pausable pauses transfers, minting, and burning through _update; it does not inherently pause approve or permit. In this contract, that means an attacker can collect approvals or signed permits while transfers are paused, then execute transferFrom once unpaused.

Remediation:

Override approve, _approve, and/or permit-reachable approval logic with whenNotPaused if the intended emergency mode is a full token-state lockdown.

2. Single-step ownership transfer and permanent renounce risk [Fixed ✅]

Issue:

transferOwnership immediately assigns owner = newOwner, and renounceOwnership permanently sets owner = address(0).

Remediation:

Use Ownable2Step, disable renounce unless explicitly required, and place high-risk admin operations behind a timelock/multisig.

LOW

No low severity issues were found.

INFORMATIONAL

No informational severity issues were found.

Technical Findings Summary

Findings

Vulnerability Level	Total	Pending	Not Apply	Acknowledged	Partially Fixed	Fixed
🔴 High	2			1		1
🟠 Medium	2					2
🟡 Low						
🟢 Informational						

Assessment Results

Score Results

Review	Score
Global Score	**85/100**
Assure KYC	Not completed
Audit Score	85/100

The Following Score System Has been Added to this page to help understand the value of the audit, the maximum score is 100, however to attain that value the project must pass and provide all the data needed for the assessment. Our Passing Score has been changed to 84 Points for a higher standard, if a project does not attain 85% is an automatic failure. Read our notes and final assessment below. The Global Score is a combination of the evaluations obtained between having or not having KYC and the type of contract audited together with its manual audit.

Audit PASS

Following our comprehensive security audit of the token contract for the **PalladiumV3** project, the project did not fulfill the necessary criteria required to pass the security audit. **After the development team's review, all critical vulnerabilities have been addressed/reviewed, and the audit results are satisfactory.**

Disclaimer

Assure Defi has conducted an independent security assessment to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the reviewed code for the scope of this assessment. This report does not constitute agreement, acceptance, or advocating for the Project, and users relying on this report should not consider this as having any merit for financial Token in any shape, form, or nature. The contracts audited do not account for any economic developments that the Project in question may pursue, and the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude, and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are entirely free of exploits, bugs, vulnerabilities or deprecation of technologies. All information provided in this report does not constitute financial or investment in Token, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence, regardless of the findings presented. Information is provided ‘as is, and Assure Defi is under no covenant to audit completeness, accuracy, or solidity of the contracts. In no event will Assure Defi or its partners, employees, agents, or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions or actions with regards to the information provided in this audit report. The assessment of Tokens provided by Assure Defi are subject to dependencies and are under continuing development. You agree that your access or use, including but not limited to any Tokens, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies with high levels of technical risk and uncertainty. The assessment reports could include false positives, negatives, and unpredictable results. The Token may access, and depend upon, multiple layers of third parties.